MATH300: Mathematical Computing · Fall 2026

Project: Chaos in Discrete Maps

Groups must be formed, and project titles submitted on Canvas, by **October 26, 2026**. The project has two graded deliverables and an oral exam: a 3–5 page report for Project 2 (due **November 20, 2026**), a poster for the final project (due **December 11, 2026**), and an oral exam held between November 30 and December 17, 2026, 3:30 pm. Read the general rules below before you start; they apply throughout the project.

General remarks

- Projects are set for a group of **two or three** students.
- Form your group by **October 26, 2026**, and submit your project title as a group assignment on **Canvas** by the same date.
- Every group member must carry an **equal portion** of the coding and of the report and poster preparation.
- The scope can be adjusted to the number of participants, their interests and prior knowledge.
- Only **Python** may be used for programming.
- Implementation, documentation, validation and demonstration examples are all part of the project.
- Within these rules you are free to choose the implementation, code structure and exact content. You can discuss this with me at any time.

Version control and GitHub

Each group keeps **one GitHub repository for the whole project** (Project 2 and the final project), and every member commits from their own GitHub account.

- Use git from the very beginning. Commits should be small, consistent and clearly described. Avoid large commits.
- Each group member uploads their own code using **their own GitHub account**. Commits from a shared or borrowed account will not be credited to the other members.
- The repository name is up to your group.
- If the repository is private, add my GitHub account `say-yas` as a collaborator.
- The repository must contain a README describing the project, how to install and run the code, and the AI-use statement (see below).

Use of AI tools

AI-generated code is not accepted in any form. Code produced by a chat assistant or code generator (e.g. ChatGPT, Claude, Gemini) and copied, pasted or adapted into your project will be treated as work that is not your own.

- **Permitted:** inline coding assistance such as GitHub Copilot autocompletion while you write the code yourself, **provided its use is acknowledged**.
- **Acknowledgement:** include an AI-use statement in the README and in the report that names the tool(s) used and the parts of the code (files or functions) where they assisted. If no AI tools were used, state this explicitly.
- **Reports and posters must not be generated by AI** in any form: their text, figures and layout must be your own work. If AI tools were used for any other non-code purpose (e.g. checking grammar or searching the literature), describe this in the same statement.
- You are responsible for every line in your repository. In the oral exam you must be able to explain and modify any part of the code you submitted.

Deliverables and deadlines

Submit each deliverable following the details provided on the **Canvas** page. The written report and poster are handed in on **Crowdmark**, and the link to your GitHub repository is submitted separately on **Canvas**.

Deliverable	**Due**	**Submission**	**Notes**
Group and project title	Monday, October 26, 2026	Group assignment on Canvas	Group of two or three members; submit the project title.
Project 2 report	Friday, November 20, 2026	Report (PDF) on Crowdmark; repository link on Canvas	3–5 pages; the page limit **includes** references, figures and appendices.
Final project poster	Friday, December 11, 2026	Poster (PDF) on Crowdmark; repository link on Canvas	36 inches high by 48 inches wide (landscape). Target audience: undergraduate classmates with a basic background in math, physics or engineering.
Oral exam	Between November 30 and December 17, 2026, 3:30 pm	Individual meeting; details discussed in class	30 minutes per student. Your poster must already be submitted on Crowdmark before your exam. The last possible slot ends December 17 at 3:30 pm.

Oral exam

The oral exam is individual and lasts 30 minutes per student. It takes place between November 30 and December 17, 2026, 3:30 pm. The oral exams and how to arrange them will be discussed in class before the Thanksgiving break.

Before your oral exam, your poster must also be submitted on **Crowdmark**; that is, you must have finished both Project 2 and the final project. In the meeting you will:

- Present your results and algorithms.
- Possibly live-code and run parts of your algorithms to generate outputs on the spot.
- Answer questions about the theory, your implementation and your results. Each member should be able to explain the whole project, not only their own parts.

Bring a laptop with the repository cloned and a working environment, so that the code runs without additional setup.

Grading

Component	Points
Group and project title (on time)	10
Project 2 report	100
Final project poster	100
Oral exam (individual)	50
Bonus: extra steps beyond the required tasks	up to 10

The Project 2 report and the poster are each graded out of 100. What matters most is:

- **Coherence:** a clear story from the question, through the methods, to the results and conclusions.
- **Correctness:** of the mathematics, the code and the results.
- **Depth of analysis and discussion:** explain what your results mean, why they look the way they do, and what their limitations are, rather than only showing them.
- **Professional presentation:** figures with labeled axes and readable font sizes, captions and legends, a clean and readable layout, and proper references.

A complete, correct and well-discussed solution of the required tasks can earn full marks. Tasks marked **(Bonus)** and the suggestions under "Going further" are optional; exploring such extra steps can earn **up to 10 bonus points** in total.

Requirements for the report and the poster

Both the report and the poster must contain a clear description of the project, references, properly labeled figures and an author-contributions statement.

Content

- A clear and concise description of the problem, the methods and the results.
- The AI-use statement described above.
- The report should not contain any significant amount of code; refer to the repository instead.

Poster

- The poster must be **self-contained**: a reader should be able to follow it without the report, the code or a spoken explanation.
- You may be selective in what you show and how you narrate your story, so that an undergraduate audience with a basic background in math, physics or engineering can follow it.

- You may choose your own title for the poster; be creative.

References

- Include a list of references at the end of the report and the poster.
- Cite sources clearly where they are used in the text (papers, books, software, code repositories and data sets).
- Use one consistent citation style throughout.

Figures

- Every plot must have labeled x and y axes (and z axis for 3D plots or color bars), including units where applicable.
- Axis labels, tick labels and legends must have a font size similar to the surrounding text, so they remain readable without zooming.
- Each figure needs a legend when it shows more than one data series, and a caption that states what is shown.
- Use log scales where appropriate (e.g. for convergence plots) and say so in the caption.

Author contributions

Each group must include an author-contributions statement in the report and the README that states:

- Who did what: which parts of the code, experiments, analysis, writing and poster each member worked on.
- To what extent: the approximate share of each member in each part (e.g. 50 % / 50 %).
- Who took the lead in which parts.

The coding and the report and poster preparation must be shared equally between the members. The commit history should be consistent with the stated contributions.

Codebase requirements

- Well structured and documented, both in the code (docstrings, comments) and in the repository (README).
- Validated: show that your implementation produces correct results, e.g. on examples with known solutions.

Project description

Goal

Very simple rules can produce very complicated behavior. In this project you will study *discrete maps*: rules that take the current state of a system and give its state at the next time step. You

will choose **at least 3 maps** whose time domain is discrete from the list in [1], implement them in Python, and explore their behavior numerically: their trajectories, how they depend on the parameters, and when they become chaotic.

Background

A discrete map is a function f that is applied again and again,

$$x_{k+1} = f(x_k), \qquad k = 0, 1, 2, \ldots,$$

starting from an initial state x_0. The state can be a single number or a point in the plane. The sequence $x_0, x_1, x_2, \ldots$ is called a *trajectory* (or orbit). A classical example is the logistic map [2],

$$x_{k+1} = r\, x_k(1 - x_k), \qquad 0 \le x_k \le 1,$$

a simple model for population growth. For small values of the parameter r the trajectory settles to a fixed value; as r increases it starts to oscillate between 2, then 4, then 8 values (period doubling), and for larger r it becomes *chaotic*: it never repeats, and two trajectories that start very close together soon become completely different. A two-dimensional example is the Hénon map [3],

$$x_{k+1} = 1 - a\, x_k^2 + y_k, \qquad y_{k+1} = b\, x_k,$$

whose trajectories are attracted to a fractal-shaped set, a *strange attractor*.

Two simple pictures help to see the structure hidden in a trajectory.

The *phase space* of a map is the set of all its possible states: the interval $[0, 1]$ for the logistic map, or the plane of all points (x, y) for the Hénon map. A *phase portrait* is a picture of trajectories drawn in phase space. Time is not shown on any axis; instead, each state (x_k, y_k) of a trajectory is drawn as one point, so the trajectory appears as a set of dots. For a two-dimensional map you plot y_k against x_k for all k, often for several initial states and usually after discarding the first iterations (the transient). The picture then shows where trajectories end up in the long run: a single point (a stable fixed point), a few points visited in turn (a periodic orbit), a closed curve, or a scattered, fractal-looking set (a strange attractor, as for the Hénon map). For maps without an attractor, such as the standard map, it shows which regions are regular and which are chaotic.

A *time-delay embedding* does the same with a single sequence: it plots $x_{k+\tau}$ against x_k for a delay τ (for example $\tau = 1$), or the three-dimensional points $(x_k, x_{k+\tau}, x_{k+2\tau})$. A trajectory that looks random in the time series often turns out to lie on a simple curve or a fractal set in such a plot. A good introduction to all of these ideas is [4].

Tasks for Project 2 (report, due November 20, 2026)

1. Choose **at least 3 maps** with a discrete time domain from [1]. Include at least one one-dimensional map (such as the logistic map or the tent map) and at least one two-dimensional map (such as the Hénon map or the standard map). Describe each map and the meaning of its parameters, and cite your sources.
2. Implement a general function that iterates a given map from an initial state for a given number of steps, and implement your chosen maps with it.
3. For each map, plot trajectories for a few parameter values: the time series x_k against k, and a phase portrait (for the two-dimensional maps) or a time-delay embedding (for the one-dimensional maps).

4. **(Bonus)** Make three-dimensional time-delay embeddings $(x_k, x_{k+\tau}, x_{k+2\tau})$ and compare different delays τ.
5. **(Bonus)** Study more than 3 maps.

Tasks for the final project (poster, due December 11, 2026)

1. For each map, choose one parameter and show how the behavior changes as you vary it: make a sequence of phase portraits (two-dimensional maps) or time-delay embeddings (one-dimensional maps) for several parameter values, from regular to chaotic behavior. Describe what changes.
2. Discuss what you observe: where is each map regular and where is it chaotic, and what do the maps have in common? What are the limitations of studying chaos numerically (finite precision, transients, finite number of iterations)?
3. **(Bonus)** For a few parameter values in the chaotic range, plot a histogram of the values visited by a long trajectory. Which values does the trajectory visit most often?
4. **(Bonus)** Estimate the parameter value at which the behavior becomes chaotic, and explain how you found it.

Going further

These optional steps count toward the bonus points. For example, you can draw a *bifurcation diagram*: for each value of a parameter (horizontal axis), iterate the map, discard the transient, and plot the values the trajectory visits afterwards. It shows period doubling as branches that split in two, and chaos as a cloud of dots. You can also study more maps from the list.

References

[1] Wikipedia. *List of chaotic maps*. https://en.wikipedia.org/wiki/List_of_chaotic_maps

[2] R. M. May (1976). Simple mathematical models with very complicated dynamics. *Nature*, 261, 459–467. https://doi.org/10.1038/261459a0

[3] M. Hénon (1976). A two-dimensional mapping with a strange attractor. *Communications in Mathematical Physics*, 50, 69–77. https://doi.org/10.1007/BF01608556

[4] S. H. Strogatz (2015). *Nonlinear Dynamics and Chaos*, 2nd edition. Westview Press.